Sliding Window Cheat Sheet (Python)

Core pattern: expand **right** pointer → grow window state → while window invalid, shrink from **left** → update answer

1. Longest Substring Without Repeating Characters

State: char frequencies **Invalid:** duplicate exists

```
def lengthOfLongestSubstring(s):
    freq = {}
    left = 0
    max_len = 0
    for right, ch in enumerate(s):
        freq[ch] = freq.get(ch, 0) + 1
        while freq[ch] > 1:
            freq[s[left]] -= 1
            left += 1
        max_len = max(max_len, right - left + 1)
    return max_len
```

Time: O(n) **Space:** O(min(n, charset))

Follow-ups

- Return the actual longest substring, not just length.
- Solve with a set instead of a dict — trade-offs?
- Generalize to "at most K repeats allowed" per char.
- Handle Unicode / multi-byte characters correctly.
- Count all substrings without repeats (not just longest).

2. Longest Substring with At Most K Distinct

State: frequency map **Invalid:** len(freq) > k

```
def lengthOfLongestSubstringKDistinct(s, k):
    freq = {}
    left = 0
    max_len = 0
    for right, ch in enumerate(s):
        freq[ch] = freq.get(ch, 0) + 1
        while len(freq) > k:
            freq[s[left]] -= 1
            if freq[s[left]] == 0:
                del freq[s[left]]
            left += 1
        max_len = max(max_len, right - left + 1)
    return max_len
```

Time: O(n) **Space:** O(k)

Follow-ups

- What changes when k = 0 or k ≥ distinct chars in s?
- Special-case k = 2 → this is Fruit Into Baskets.
- Return the substring itself, and its start index.
- Adapt for a streaming input where s isn't fully known upfront.
- Why use dict size instead of scanning freq values each time?

3. Longest Substring with Exactly K Distinct

State: frequency map **Invalid:** len(freq) > k

```
def longestSubstringExactlyKDistinct(s, k):
    freq = {}
    left = 0
    max_len = 0
    for right, ch in enumerate(s):
        freq[ch] = freq.get(ch, 0) + 1
        while len(freq) > k:
            freq[s[left]] -= 1
            if freq[s[left]] == 0:
                del freq[s[left]]
            left += 1
        if len(freq) == k:
            max_len = max(max_len, right - left + 1)
    return max_len
```

Time: O(n) **Space:** O(k)

Follow-ups

- Why does shrinking only on > k (not ≥ k) still give exactly-k windows?
- Count the number of substrings with exactly K distinct chars — use atMost(k) − atMost(k−1).
- Explain why "exactly" can't just check equality without the atMost trick for counting problems.
- What if K is larger than the alphabet size?

4. Longest Repeating Character Replacement

State: freq + maxFreq **Invalid:** (winSize − maxFreq) > k

```
def characterReplacement(s, k):
    freq = {}
    left = 0
    max_freq = 0
    max_len = 0
    for right, ch in enumerate(s):
        freq[ch] = freq.get(ch, 0) + 1
        max_freq = max(max_freq, freq[ch])
        window_size = right - left + 1
        if window_size - max_freq > k:
            freq[s[left]] -= 1
            left += 1
        else:
            max_len = max(max_len, window_size)
    return max_len
```

Time: O(n) **Space:** O(26)

Follow-ups

- Why is it safe that max_freq is never decreased when the window shrinks?
- Extend to a 52-letter (upper + lower) or full Unicode alphabet.
- Return the actual resulting string after replacement.
- What if each character has a different replacement cost?
- Relate this pattern to Max Consecutive Ones III.

5. Max Consecutive Ones III

State: zero_count **Invalid:** zero_count > k

```
def longestOnes(nums, k):
    left = 0
    zero_count = 0
    max_len = 0
    for right, num in enumerate(nums):
        if num == 0:
            zero_count += 1
        while zero_count > k:
            if nums[left] == 0:
                zero_count -= 1
            left += 1
        max_len = max(max_len, right - left + 1)
    return max_len
```

Time: O(n) **Space:** O(1)

Follow-ups

- Return the indices of the zeros that would be flipped.
- Generalize: "at most k changes of any value" in an array.
- Solve it without shrinking the window (never-shrink variant) — does the window size still equal the answer?
- Adapt to a circular array.

6. Fruit Into Baskets

State: frequency map **Invalid:** len(freq) > 2

```
def totalFruit(fruits):
    freq = {}
    left = 0
    max_len = 0
    for right, fruit in enumerate(fruits):
        freq[fruit] = freq.get(fruit, 0) + 1
        while len(freq) > 2:
            freq[fruits[left]] -= 1
            if freq[fruits[left]] == 0:
                del freq[fruits[left]]
            left += 1
        max_len = max(max_len, right - left + 1)
    return max_len
```

Time: O(n) **Space:** O(1) (≤ 3 keys)

Follow-ups

- Generalize to K baskets — identical to "At Most K Distinct" (#2).
- Return the actual subarray of fruits picked, and the basket boundaries.
- What if each basket has a capacity limit on number of fruits?
- Solve if you must also track which two fruit types were chosen.

7. Minimum Window Substring

State: frequency map **Invalid:** missing required chars

```
from collections import Counter

def minWindow(s, t):
    if not s or not t:
        return ""
    need = Counter(t)
    missing = len(t)
    left = 0
    bl, blen = 0, float('inf')
    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
        need[ch] -= 1
        while missing == 0:
            if right - left + 1 < blen:
                bl, blen = left, right - left + 1
            need[s[left]] += 1
            if need[s[left]] > 0:
                missing += 1
            left += 1
    return "" if blen == float('inf') else s[bl:bl + blen]
```

Time: O(|s| + |t|) **Space:** O(|t|)

Follow-ups

- Return all minimum windows, not just the first found.
- Make matching case-insensitive or ignore certain characters.
- Support multiple target strings (cover t1 AND t2 simultaneously).
- Allow the window to be "valid" if it's missing at most one character.
- How would you handle s being a very large stream (can't index freely)?

Complexity Summary & General Tips

#	Problem	Time	Space
1	Longest Substr. w/o Repeat	O(n)	O(min(n,Σ))
2	At Most K Distinct	O(n)	O(k)
3	Exactly K Distinct	O(n)	O(k)
4	Char Replacement	O(n)	O(26)
5	Max Consecutive Ones III	O(n)	O(1)
6	Fruit Into Baskets	O(n)	O(1)
7	Min Window Substring	O(|s|+|t|)	O(|t|)

Cross-cutting follow-ups: convert to a two-pointer / prefix-sum solution and compare; adapt for a fixed-size window vs. variable-size window; handle arrays instead of strings; make the window state thread-safe for a streaming/online setting; discuss why the amortized time stays O(n) even though there's a nested while loop (each pointer moves forward at most n times total).

Follow-Up Answers (Part 1 of 2)

Simple explanations for the follow-up questions on problems 1–4

1. Longest Substring Without Repeating Characters

Return the actual substring, not just length?

Save **best_start = left** every time you update max_len. At the end return s[best_start : best_start + max_len].

Set instead of a dict — trade-offs?

A set only says "is this char in the window," not its count, so you shrink one character at a time until the duplicate is gone. A dict/last-seen-index map can jump left straight past the duplicate, which is a bit faster in practice but same O(n) overall.

Generalize to "at most K repeats" per char?

Change the invalid condition from freq[ch] > 1 to freq[ch] > K. Everything else stays the same.

Handle Unicode / multi-byte characters?

Python 3 strings already iterate by Unicode code point, not raw bytes, so the same code works as-is — no special handling needed.

Count all substrings without repeats?

At every valid window position, every substring ending at "right" and starting anywhere from "left" to "right" is duplicate-free. So add (right - left + 1) to a running total each step.

2. Longest Substring with At Most K Distinct

What if k = 0 or k ≥ distinct chars in s?

k = 0 means no characters allowed at all, so the answer is 0. If k covers every distinct character already in s, the whole string is valid, so the answer is len(s).

Special case k = 2?

That's exactly Fruit Into Baskets — "at most 2 distinct types" is the same rule with K hardcoded to 2.

Return the substring and its start index?

Same trick as problem 1: remember "left" whenever max_len is updated.

Adapt for a streaming input?

Sliding window never looks ahead anyway — it only reacts to the current window. So you can feed characters in one at a time as they arrive and it works unchanged.

Why dict size instead of scanning freq values?

len(freq) is an O(1) lookup in Python. Scanning every value to count non-zero entries would cost O(alphabet size) on every single step, which adds needless overhead.

3. Longest Substring with Exactly K Distinct

Why shrink only on > k (not ≥ k)?

If you shrank as soon as you hit exactly k, the window could never rest at exactly k long enough to be measured. Shrinking only once you exceed k lets the window sit at (or return to) exactly k, so you catch it whenever len(freq) == k.

Count substrings with exactly K distinct chars?

Count substrings with at most K distinct, subtract substrings with at most K−1 distinct. What's left is exactly those with exactly K distinct — a standard inclusion-exclusion trick.

Why not just check equality directly for counting?

A single window position can contain many valid substrings ending at different points, so simple equality checks miss most of them. The atMost(k) − atMost(k−1) subtraction counts them all without enumerating each one by hand.

What if K is bigger than the alphabet size?

You can never reach K distinct characters, so the answer is 0.

4. Longest Repeating Character Replacement

Why is it safe that max_freq never decreases?

max_freq only needs to represent the best majority-count ever seen so far. Even if it's stale for the current window, using a larger stale value just makes the window shrink a little less aggressively — it can never cause the algorithm to report a window that's actually invalid.

Extend to 52 letters or full Unicode?

Swap the fixed 26-slot array for a dict. Everything else (including O(n) time) stays the same; space becomes O(alphabet size) instead of O(26).

Return the actual resulting string?

Once you know the best window [left, right], replace every character in it that isn't the majority character with the majority character.

What if replacement cost varies per character?

Instead of comparing window_size − max_freq to k, track the total cost of replacing the minority characters in the window and compare that total against a budget k.

How does this relate to Max Consecutive Ones III?

Max Consecutive Ones III is the same problem with only two "characters" — 0 and 1 — where you're allowed to flip up to k zeros into ones.

Follow-Up Answers (Part 2 of 2)

Simple explanations for the follow-up questions on problems 5–7, plus cross-cutting tips

5. Max Consecutive Ones III

Return the indices of the flipped zeros?
Keep a list/queue of zero positions as they enter the window. Whenever you find the best window, those saved positions (that fall within it) are the zeros to flip.

Generalize to "at most k changes of any value"?
Instead of counting zeros, count elements that don't match your target value, and use that count exactly like zero_count.

Never-shrink variant — does window size still equal the answer?
Yes. If the window becomes invalid, you simply stop growing max_len while still moving "left" forward by one. The window's size can hold steady or grow but never drops below its best-ever length, so (right - left + 1) at the end still equals the true maximum.

Adapt to a circular array?
Concatenate the array with itself and run the normal algorithm, but cap the window size at the original array's length so it doesn't wrap around more than once.

6. Fruit Into Baskets

Generalize to K baskets?
Identical to "At Most K Distinct" (#2) — just change len(freq) > 2 to len(freq) > K.

Return the actual subarray and basket boundaries?
Same pattern as before: remember "left" whenever max_len updates. The subarray is fruits[left : left + max_len].

What if each basket has a capacity limit?
Add a second invalid condition: also shrink the window if any single fruit type's count exceeds the basket's capacity, in addition to shrinking when there are more than 2 distinct types.

Track which two fruit types were chosen?
Just read the keys of the freq dict at the best window — there will be at most 2, and those are the chosen basket types.

7. Minimum Window Substring

Return all minimum windows, not just the first?
Once you know the minimal length from a first pass, do a second pass (or keep collecting during the first) and record every window that matches that exact minimal length instead of stopping at the first.

Case-insensitive or ignore certain characters?
Lowercase both s and t before building the Counter, or strip/skip the ignored characters before comparing.

Support multiple target strings (t1 AND t2)?
Merge the required counts of t1 and t2 into one combined Counter (taking the max or sum needed per character) and run the same algorithm on the merged requirement.

Window valid if missing at most one character?
Instead of requiring missing == 0 to enter the shrink loop, allow missing ≤ 1, adjusting the loop condition to match.

Handle s as a very large stream?
Keep only a bounded buffer of the current window's characters (you never need anything before "left"), and process each new character as it arrives instead of indexing into a fixed array.

Cross-Cutting Follow-Ups

Two-pointer / prefix-sum comparison?
Sliding window already *is* a two-pointer technique. Prefix sums are great for range-sum queries but don't directly help with "is this window valid" conditions like distinct-character counts.

Fixed-size vs. variable-size window?
Fixed-size: both pointers move together every step, so the window size never changes. Variable-size: right grows freely, and left only moves when the window becomes invalid.

Arrays instead of strings?
The exact same code works — Python indexes and iterates lists just like strings. The frequency map just keys on array elements instead of characters.

Thread-safe for streaming/online use?
Wrap left, freq, and friends inside a class with a "process next element" method. You only need locks if multiple threads mutate the same state object concurrently.

Why is it still O(n) despite the nested while loop?
"left" only ever moves forward, never backward, and it can move at most n times in total across the entire run — no matter how many times the outer loop advances "right." So the total work done by the inner while loop, summed over the whole algorithm, is bounded by n.